

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №4 (Тимус)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 4-ого блока Тимуса

Задача №1 «1162. Currency Exchange»

Описание

В городе есть несколько обменных пунктов. Каждый пункт работает с парой валют и задает курс обмена в обе стороны, а также комиссию, которая вычитается из исходной валюты перед обменом. У Ника есть сумма V в валюте S . Нужно определить, можно ли с помощью последовательности обменов увеличить количество денег и снова иметь деньги в валюте S .

Ограничение времени: 1 секунда.

Ограничение памяти: 64 МБ.

Решение

Будем хранить для каждой валюты максимальную сумму, которую можно получить в этой валюте. Изначально известна только сумма V в валюте S . Каждый обменный пункт дает два ориентированных ребра. Если в валюте a сейчас есть сумма $d[a]$, комиссия равна c , а курс равен r , то после обмена можно получить $(d[a] - c) \cdot r$ в валюте b , но только если $d[a] > c$.

Дальше делаем обычные релаксации, как в алгоритме Беллмана–Форда, только вместо минимального расстояния улучшаем максимальную сумму. Если на N -м проходе еще получилось что-то улучшить, значит существует выгодный цикл обменов, и ответ YES. Иначе ответ NO.

Доказательство

После первой серии релаксаций массив d хранит лучшие суммы, которые можно получить не более чем за один обмен. После второй — не более чем за два обмена, и так далее. Это верно по индукции: очередная релаксация пробует добавить один обмен к уже найденной последовательности.

Если после $N - 1$ обменов все улучшения уже закончились, то выгодного цикла нет: все лучшие значения достигаются простыми путями. Если же на N -м проходе улучшение все еще возможно, то в использованной цепочке есть повторная валюта, а значит есть цикл, на котором сумма растет. Такой цикл можно использовать для увеличения капитала, поэтому ответ положительный.

Алгоритмическая сложность

- **По времени:** $O(NM)$, так как делается до N проходов по ребрам, а из каждого пункта получается два ребра.
- **По памяти:** $O(N + M)$.

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct Edge {
6     int a, b;
7     double r, c;
8 };
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     int n, m, s;
15     double v;
16     cin >> n >> m >> s >> v;
17
18     vector<Edge> e;
19
20     for (int i = 0; i < m; i++) {
21         int a, b;
22         double rab, cab, rba, cba;
23         cin >> a >> b >> rab >> cab >> rba >> cba;
24
25         e.push_back({a, b, rab, cab});
26         e.push_back({b, a, rba, cba});
27     }
28
29     vector<double> d(n + 1, 0.0);
30     d[s] = v;
31
32     for (int i = 0; i < n; i++) {
33         bool ok = false;
34
35         for (auto x : e) {
36             if (d[x.a] > x.c) {
37                 double val = (d[x.a] - x.c) * x.r;
38                 if (val > d[x.b] + 1e-9) {
39                     d[x.b] = val;
40                     ok = true;
41                 }
42             }
43         }
44
45         if (ok && i == n - 1) {
46             cout << "YES\n";
47             return 0;
48         }
49     }
50
51     cout << "NO\n";
52     return 0;
53 }
```

Задача №2 «1450. Российские газопроводы»

Описание

Дана сеть направленных газопроводов. Каждый газопровод идет из станции A_i в станцию B_i и приносит прибыль C_i . Известно, что если из станции X можно попасть в станцию Y , то обратно из Y в X попасть нельзя, то есть циклов в графе нет. Нужно найти максимальную прибыль маршрута из станции S в станцию F или вывести **No solution**, если такого маршрута нет.

Ограничение времени: 0.5 секунды.

Ограничение памяти: 64 МБ.

Решение

Так как в графе нет циклов, сначала получим топологический порядок вершин. В таком порядке все ребра идут слева направо, поэтому к моменту обработки вершины уже известна лучшая прибыль пути до нее.

Будем хранить $d[v]$ — лучшую прибыль пути из S в вершину v . Изначально $d[S] = 0$, остальные значения считаем недостижимыми. Затем идем по вершинам в топологическом порядке и для каждого ребра $v \rightarrow to$ пробуем улучшить $d[to]$ значением $d[v] + cost$. В конце посмотрим значение $d[F]$.

Доказательство

В топологическом порядке все предки вершины идут раньше нее. Поэтому когда алгоритм начинает обрабатывать вершину v , все возможные последние ребра пути в v уже были рассмотрены, а значит $d[v]$ содержит максимальную прибыль пути из S в v . Поэтому если F достижима, $d[F]$ будет максимальной прибылью, иначе значение останется недостижимым.

Алгоритмическая сложность

- По времени: $O(N + M)$.
- По памяти: $O(N + M)$.

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int n, m;
10    cin >> n >> m;
```

```

11
12 vector<vector<pair<int, int>>> g(n + 1);
13 vector<int> deg(n + 1, 0);
14
15 for (int i = 0; i < m; i++) {
16     int a, b, c;
17     cin >> a >> b >> c;
18     g[a].push_back({b, c});
19     deg[b]++;
20 }
21
22 int s, f;
23 cin >> s >> f;
24
25 queue<int> q;
26 for (int i = 1; i <= n; i++) {
27     if (deg[i] == 0) {
28         q.push(i);
29     }
30 }
31
32 vector<int> order;
33 while (!q.empty()) {
34     int v = q.front();
35     q.pop();
36     order.push_back(v);
37
38     for (auto x : g[v]) {
39         int to = x.first;
40         deg[to]--;
41         if (deg[to] == 0) {
42             q.push(to);
43         }
44     }
45 }
46
47 long long bad = -(long long)4e18;
48 vector<long long> d(n + 1, bad);
49 d[s] = 0;
50
51 for (int v : order) {
52     if (d[v] == bad) {
53         continue;
54     }
55
56     for (auto x : g[v]) {
57         int to = x.first;
58         int cost = x.second;
59         d[to] = max(d[to], d[v] + cost);
60     }
61 }
62
63 if (d[f] == bad) {
64     cout << "No solution\n";
65 } else {
66     cout << d[f] << '\n';
67 }
68
69 return 0;
70 }

```